Generative Specification — A Field Guide

How to give an AI agent the discipline it was already trained on — but won't apply unless you make it.

Author: Juan Carlos Ghiringhelli (PragmaWorks)

---

1. The theory

The reader is stateless

An AI agent can generate an entire system in one session. To do it, it silently resolves a thousand implicit decisions — units, field ordering, layer ownership, error semantics — drawing on how such systems are *usually* built. Each choice is locally reasonable; across sessions, teams, and services they diverge. This is **architectural drift**.

The cause is not a weak model. It is that the model is a **stateless reader**: it begins every session with no memory of prior decisions, no shared context, and no ability to ask. Everything you did not write down is, to it, absent. So the design target is precise: **a specification from which a reader with no context can derive the correct output, alone.** That property — *derivability by a stateless reader* — is the whole discipline.

Why specifying works: the bridge, and its asymmetry

Every structural discipline you already use — naming, SOLID, a schema, a domain language, a contract — is a **bridge between human meaning and executable code**. It carries intent across the gap. Those bridges were built for the next *human* reader.

The transformer is the **first reader trained on both banks**: the corpus of human language and the corpus of code. It crosses the bridge in both directions.

And the bridge is **asymmetric** — this is the leverage. Training is overwhelmingly natural language; code is a small, exact, unforgiving slice where one wrong token breaks everything. The model is therefore *far stronger on human meaning than on exact code*. Encoding intent as structure — names, contracts, constraints in human-conceptual terms — **routes the hard half of the problem through the model's strong half.** That is the mechanism: specifying moves the load to the bank the model is fluent on.

Restriction is activation — take your AI to school

The model already went to school. Its training contains the entire formal tradition — Hoare logic, type theory, design by contract, SOLID, DDD, REST, deontic constraints. **It does not lack the knowledge; it lacks the instruction to apply it.**

So every constraint you name in a specification is not teaching — it is **activation**. Naming "hexagonal architecture" or "idempotent handler" or "per-tenant isolation" rules out the wrong programs and selects the correct one the model already knew how to write. A discipline of removal, in Robert Martin's sense: you do not add capability, you *delete the freedom to be wrong*. What remains is the program that was always latent in the model's training.

Generative Specification is how you make a model that has been to school actually do the coursework.

---

2. What to do

One door: the sentinel navigational tree

Give the agent a single entry point that declares scope and routes to the slice each task needs — not forty files to guess among. A well-formed sentinel carries five things:

```
# CLAUDE.md (the sentinel — the one door)

## Identity        What this system is (screaming architecture — the name states the domain)
## Standards       The disciplines in force (SOLID, TDD, hexagonal, conventional commits)
## Constraints     Inviolable rules and forbidden patterns (each tied to a real past incident)
## Tool sequencing WHEN to prefer which tool — not just a list          ← most often missing
## Routing         Where each concern lives → links to the scoped child specs
```

Tool sequencing is the single most common gap: a tree that lists tools but never says *when* to prefer one forces the agent to guess — and guessing is where drift enters.

Keep the door small. The agent re-reads the sentinel every turn, and attention degrades with depth — a rule on line 400 is effectively invisible by turn 50, and long files truncate silently (~300 lines is the safe ceiling per file). Hold `CLAUDE.md` near 250–300 lines: *name* the disciplines (SOLID, TDD), don't tutorialize them — the model already went to school. When a *spec* outgrows ~500 lines, don't load it whole; build a ~50-line **spec-map** that points each task to the exact line ranges it needs. In one brownfield task this cut spec context ~82% (55k → 9.9k tokens) while raising expected fix correctness. *(pilot)*

Two logs the door must carry — each turns a one-time correction into permanent grammar: - **Corrections Log** — when you tell the agent *"don't do that"* about a pattern it produced, it appends a dated one-liner (`[2026-03-12] — handle invalid cases with early-return guards, never nested conditionals`). It reads the log next session and won't repeat the deviation. - **Known Pitfalls** — technology traps, not behavioral ones: the flag that silently no-ops, the library whose types lie about runtime. Three parts each — what goes wrong, the wrong pattern, the right one.

The stateless reader can't remember it hit the trap yesterday; these are how the repo remembers *for* it — the ratchet, pointed at the sentinel.

Grade the spec the way an AI reads it: seven properties

Seven properties, each named for a real production failure, each scored **0–2** for a 14-point total. Run it on any repo today:

#	Property	Ask of the spec	Removes
1	**Self-describing**	Does each file announce its purpose and domain?	Hidden intent the reader must infer
2	**Bounded**	Can a task load only its slice, not the whole system?	An unbounded surface nobody can scan
3	**Verifiable**	Is "done" defined by passing gates, not "it compiled"?	Unchecked correctness
4	**Defended**	Are rules *enforced* (hooks/CI), not advisory?	Rules the model treats as optional
5	**Auditable**	Is the *why* recorded (ADRs, commits)?	Lost rationale
6	**Composable**	Can a unit be understood and changed in isolation?	Tangled coupling
7	**Executable**	Are contracts run against a live system, not assumed?	Specs never tested against reality

Self-describing and **Bounded** carry most of the weight: a bounded, self-describing spec activates the model's *relevant* knowledge instead of its full prior.

What the rubric does not measure — specify it anyway. Architectural correctness and supply-chain safety are orthogonal. A repo can score full marks on all seven properties — perfect layers, full enforcement, complete audit trail — and still ship high-severity CVEs pulled in by an unconstrained dependency chain. The rubric grades how the AI *structured* what it produced, not what it *selected*. So state selection as a constraint too: `npm audit` zero-HIGH as a P1 gate, plus an approved/forbidden library list. An executor handed no dependency policy is unconstrained in the supply-chain dimension — and will act like it.

Six pathologies you'll recognize — and the property that prevents each

The rubric grades structure; the pathologies name what its absence *feels like* in a real repo. Practitioners recognize their own problems here — that recognition is the hook. Each is one or more absent properties made concrete:

Pathology	What you'll recognize	Property that prevents it
Session Amnesia	the model repeats a decision you already corrected	**Auditable** — the correction is on record, so the next session inherits it
Implicit Contract Syndrome	two systems agreed on nothing (the Mars Orbiter problem)	**Executable** — behavioral contracts run against the live boundary
Specification Absence	no sentinel, no architecture decision on record	**Bounded** — the sentinel bounds and routes the reader's context; without it, everything must be scanned
Implicit Architecture	the structure lives in someone's head	**Self-describing** — screaming architecture: naming and layering announce the structure, so it is never inferred
AI Security Blindspot	the supply chain is ungoverned	*none of the seven — the rubric misses this; spec it separately* (`npm audit` gate + approved-library list)
Test Theater	high line coverage, low mutation score	**Verifiable** — mutation measures what was *caught*, not what ran

The loop: retrieve → generate → verify

Author the structure so the agent **retrieves** context instead of re-deriving it; the agent **generates**; the harness **verifies** — the full test pyramid (unit, integration, E2E, mutation, contract, SLO) plus AI-as-QA run against the *live* application, not assumed from a clean compile. The verify step is **generative execution**: the agent operates the real machine — runs the tests, hits the endpoints, reads the logs — and checks output against the specification.

Why the verify step insists on mutation testing. An AI that writes its own tests *knowing the implementation* will write them to pass, not to catch. Line coverage rewards exactly that: a suite that executes every line but asserts nothing scores 100% coverage and 0% mutation score. In one project an 80%-line-coverage suite scored **58%** under Stryker — 22 points of tests that ran the right code and checked nothing. Run mutation *right after each test batch*, not at release; the surviving mutants are the assertions you forgot to write. Coverage measures what was executed; mutation measures what was *caught*.

Every defect becomes a permanent test and a named rule. This is **the ratchet**: the rule set only grows, and each fixed bug makes its class of failure unreachable. A defect is not "the method failed" — it is a **specification query**: *what constraint, had it been written, would have ruled this out?*

The horizon: what you stop doing

Why bother building all this structure? Because each tier it unlocks removes a whole class of work from your hands:

- **T1** — you don't write the code, and you don't review what was generated; the harness certifies it.
- **T2** — you don't touch deployment; the spec drives CI/CD.
- **T3** — you don't monitor; the spec's behavioral contracts run against the live system.

Each tier is admissible only when the one before it holds. (The full cascade runs to T6; the treatment is in the Compendium.) The horizon is not "the AI writes code faster" — it is that specifying correctly is the only thing left that you do.

The sharpest move: prescriptive, not descriptive

The most common objection is *"the agent cuts corners."* It is real — and it is a specification problem. Under speed-and-token pressure, a **descriptive** spec ("build a rate limiter") lets the model floor to the literal minimum. A **prescriptive** spec — intent made explicit ("reject the 101st request in a 60s window with HTTP 429, per API key, return `Retry-After`") — closes the output space. What the spec does not close, the agent is free to floor.

Reach for the RFC 2119 keywords — they are the lexical tool for closing a degree of freedom. Phrase each load-bearing obligation with a capitalized normative word: **MUST** closes the freedom outright (a blocking gate), **SHOULD** is defeasible (a warning — deviation needs a recorded reason), **MAY** is ungated (permitted, unchecked). The keyword sets both the obligation and the gate's severity, so "MUST reject the 101st request in a 60-second window per API key with HTTP 429" closes what "the rate limiter should handle bursts" leaves open. Every MUST is an acceptance criterion, which makes it a machine-checkable probe — the keyword is where a prescriptive clause connects to **Verifiable** and the verify loop. Keyword the obligations that carry weight, not every sentence; over-marking is harness excess.

The unit of work: a bound prompt, not a task title

A roadmap line like *"build the connection system"* forces the agent to reconstruct scope at execution time — exactly where it invents. Bind every task to a prompt that carries its own references, scope, and acceptance test:

```
## [ID] — [task name]
**Load:** [exact artifacts to read — and what NOT to load]
**Scope:** [what to build — and, explicitly, what NOT to touch]
**Acceptance:**
- [ ] [specific, verifiable criterion]
- [ ] full suite passes
- [ ] exercised at the HTTP/CLI boundary
**Commit:** feat(scope): [description]
```

The load-bearing line is **what NOT to touch**. Facing a failing test, an agent's path of least resistance is to edit the production code until the test passes; a `NOT IN SCOPE: implementation code` line makes that path unreachable. It is the prescriptive move applied to a whole session.

---

3. The numbers

What the discipline buys. Each result is committed, reproducible evidence — *how* each was produced is in the paper and the linked experiments.

- **Structure** — naive prompting scored **3/14** on the rubric; GS-structured output reached **14/14**, and held even when the harness was *tool-generated*. *(measured)*
- **Retrieval cost** — authored structure costs **up to 3× fewer tokens per query at higher accuracy** than dumping context or searching code at query time. *(measured)*
- **Model independence** — a mid-tier model matched a frontier model **149/149 at ≈6× lower cost**: the effect is a property of the specification, not the model. *(pilot)*
- **Formal tier** — a compiler derived from its own specification, **386/386** acceptance tests. *(measured)*
- **Production** — a service specified, generated, and deployed to a live runtime: **13/13** behavioral probes (1,013 assertions), **6/6** SLO gates. *(measured)*
- **Reproducibility** — **104 tests** regenerated from a committed spec by an independent third party. *(measured)*
- **Corner-cutting** — a prescriptive spec moved the same task from **0/3 → 3/3** against a held-out oracle at equal token cost. *(pilot)*

Not claimed. There is no measured end-to-end "your token bill drops X%"; the binding metric is *tokens-per-correct-output*, not tokens spent. Leading with these limits is deliberate — it is why the measured results above can be trusted.

The other side, measured by someone else. Independent work shows what happens *without* authored structure: Orlanski et al.'s SlopCodeBench (2026) instruments agent trajectories that extend their own prior solutions and finds structural erosion rising in 80% of them, with agent code running 2.2× more verbose than matched human-authored code and deteriorating each iteration while human code stays flat — the failure mode GS is built to prevent, measured by a group that never tested GS.

---

4. Start here — this week

1. **Create the sentinel** (`CLAUDE.md` at the repo root) with the five categories. One door.
2. **Write the architectural constitution** before the first agent session — identity, layers and their ownership, the schema, a skeleton decision record.
3. **Turn on the harness** — hooks + CI that gate on tests, types, and lint. "Done" = gates pass. **ForgeCraft** installs these quality gates in CI — the **Defended** property made installable.
4. **Grade yourself** on the seven-property rubric. Your lowest two scores are your next two moves. `npx pragmaworks audit` runs the seven-property rubric automatically; **CodeSeeker** (hybrid graph search) makes the *retrieve* step operational so the agent traverses structure instead of re-deriving it.

Hooks aren't just safety — they're budget. Every check that runs as a hook costs zero context tokens; the same check done in-conversation — compile, run tests, scan for forbidden patterns — costs a thousand-plus tokens *each time the agent redoes it by hand*. Move verification into hooks and the freed budget goes to work instead of re-checking. And **Defended** scores 0 until hooks actually run: *"add pre-commit hooks"* written in a status file is not a defended system; hooks logging real violations are.

What GS does not remove — the judgment layer. Naming what stays human is what makes the promise credible. GS automates specification, generation, and verification; it does not touch *domain expertise*, the *strategic decision about what should exist*, real *user research*, *aesthetic judgment*, or *compliance sign-off*. Those are the terminus every tier routes toward, not the work the harness absorbs. If a pitch claims the machine decides what to build, it is overselling; GS lowers the cost of everything downstream of that decision so the decision is all that is left.

The specification is the mold. The AI is the foundry. The scarce resource — the one that does not regenerate for free — is the judgment to specify correctly.

Links

- **White paper** (the full argument and citations): `github.com/jghiringhelli/generative-specification` → `docs/white-paper/GenerativeSpecification_WhitePaper.md`
- **Compendium** (complete evidence, the 29-pathology catalog, the formal treatment): same repo, `docs/white-paper/GenerativeSpecification_Compendium.md`
- **Experiments** (per-run JSON evidence — AX, EX, KX, RX, MX, RND-1): `github.com/jghiringhelli/generative-specification/tree/main/experiments`
- **Formal-tier experiment (ALX)**: `github.com/jghiringhelli/loom/tree/main/experiments/alx`
- **Run it on your team's codebase**: `pragmaworks.dev`